

■ Kolokwium Java OOP

Przewodnik dla zielonego — od zera do kolokwium

Analiza kolokwiów 2021 · 2022 · 2023 · 2024

Dla kogo?	Studentzi, którzy znaj■ Jav■ podstawowo ale nie s■ pewni wzorców OOP
Co zawiera?	Wszystkie wzorce z poprzednich egzaminów + gotowe snippety do kopiowania
Jak czyta■?	Przejrzyj wszystko wieczór przed egzaminem, na egzaminie wró■ do konkretnego działu

1. Setup — pierwsze 15 minut na egzaminie

Masz 15 minut na pobranie czegośkolwiek. To jest twój priorytet. Wykonaj kolejno te kroki — nie zmieniaj kolejności.

1	Pobierz repo Pobierz kolokwium_repo.zip z wcześniej przygotowanego miejsca (np. GitHub, Google Drive, pendrive). Rozpakuj do katalogu na pulpicie.
2	Otwórz IntelliJ File → Open → wybierz plik pom.xml z rozpakowanego katalogu. Kliknij Open as Project .
3	Poczekaj na Maven IntelliJ automatycznie pobierze zależności (JUnit, Jackson). Widzisz pasek postępu na dole. Zajmuje ~1 minutę przy dobrej sieci.
4	Przeczytaj treść kolokwium Masz ~10 minut. Przeczytaj całość. Zanotuj sobie: ile klas, jaki wyjątek, czy są testy JUnit, czy jest plik SVG.
5	Otwórz CHEATSHEET.md W IntelliJ otwórz CHEATSHEET.md obok edytora kodu. Szukaj wzorca pasującego do kroku, który aktualizujesz.

■ ■ ■ Jeśli Maven nie pobiera zależności — sprawdź czy internet działa. Jeśli nie — możesz ręcznie skopiować pliki JAR (JUnit, Jackson) i dodać je w File → Project Structure → Libraries.

2. Wzorzec kolokwium — co powtarza się co roku

Po analizie wszystkich 4 kolokwίων widać wyraźny schemat. Każde kolokwium jest inna tematycznie (COVID, ceny, mapa, zegar), ale **wzorzec kodu jest identyczny**.

Krok	Co robisz	Plik w repo
1–2	Klasa abstrakcyjna + 2 podklasy	AbstractBase.java
2–3	fromCsv() — fabryka wczytująca plik CSV	CsvParser.java
3	Własny wyjątek (checked lub unchecked)	ExceptionTemplates.java
4–5	Struktury danych w podklasach + uzupełnienie fromCsv	CsvParser.java
6	Przeciążenie metody (inna sygnatura)	AbstractBase.java
7–8	Metoda abstrakcyjna + sortowanie listy	SortingExamples.java
9+	Zapis do pliku / testy JUnit / Jackson XML / SVG	FileIOExamples.java ExampleTest.java

■ Zawsze czytaj CAŁY treść kolokwium przed pisaniem. Kroki są propozycją kolejności — możesz zacząć od czegokolwiek.

3. Klasa abstrakcyjna i dziedziczenie

To fundament każdego kolokwium. Schemat jest zawsze taki sam: jedna klasa abstrakcyjna (nadrzędna) i dwie klasy konkretne (podrzędne).

3.1 Co to jest klasa abstrakcyjna?

Klasa abstrakcyjna to klasa, której **nie możesz tworzyć bezpośrednio** (nie możesz zrobić `new KlasaAbstrakcyjna()`). Służy jako wspólna baza dla podklas. Może mieć metody abstrakcyjne (bez ciała) — podklasy muszą je zaimplementować.

```
// KLASA ABSTRAKCYJNA – wspólna baza
public abstract class Country {
    private final String name;    // prywatne + final = niezmiennie

    protected Country(String name) {    // konstruktor protected
        this.name = name;
    }

    public String getName() {        // akcesor (getter)
        return name;
    }

    // Metoda abstrakcyjna – podklasy MUSZĄ ją zaimplementować
    public abstract int getDeaths(LocalDate date);
}

// PODKLASA BEZ PROWINCJI
public class CountryWithoutProvinces extends Country {
    // Konstruktor wywołuje konstruktor nadklasy
    public CountryWithoutProvinces(String name) {
        super(name);    // super() = konstruktor Country
    }

    @Override
    public int getDeaths(LocalDate date) {
        return /* wartość z mapy */ 0;
    }
}

// PODKLASA Z PROWINCJAMI
public class CountryWithProvinces extends Country {
    private final CountryWithoutProvinces[] provinces;

    public CountryWithProvinces(String name, CountryWithoutProvinces[] provinces) {
        super(name);
        this.provinces = provinces;
    }

    @Override
    public int getDeaths(LocalDate date) {
        int sum = 0;
        for (var p : provinces) sum += p.getDeaths(date);
        return sum;    // sumuje rekurencyjnie
    }
}
```

3.2 Polimorfizm — dlaczego to jest potrzebne

Polimorfizm oznacza, że możesz trzymać obiekty różnych podklas pod typem klasy nadrzędnej. Java automatycznie wywoła właściwą implementację.

```
// Obie klasy trzymamy jako "Country"
Country c1 = new CountryWithoutProvinces("Polska");
Country c2 = new CountryWithProvinces("Australia", provinces);

// Java sama wie którą getDeaths() wywoła
c1.getDeaths(date); // wywoła CountryWithoutProvinces.getDeaths
c2.getDeaths(date); // wywoła CountryWithProvinces.getDeaths

// Lista polimorficzna
List<Country> countries = new ArrayList<>();
countries.add(c1);
countries.add(c2);
```

4. Parsowanie CSV — wczytywanie pliku danych

Każde kolokwium ma plik CSV do wczytania. Wzorzec jest zawsze podobny — różni się tylko separatory i struktura.

4.1 Podstawowy wzorzec — Scanner

```
public static Country fromCsv(Path path) throws IOException {
    try (Scanner scanner = new Scanner(path)) {
        // Krok 1: wczytaj nagłówki (pierwsza linia)
        String[] headers = scanner.nextLine().split(";", -1);

        // Krok 2: pomiń drugą linię jeśli trzeba
        scanner.nextLine();

        // Krok 3: czytaj dane linia po linii
        while (scanner.hasNextLine()) {
            String line = scanner.nextLine();
            String[] cols = line.split(";", -1);

            // Krok 4: parsuj wartości
            String date = cols[0].trim();
            int value = Integer.parseInt(cols[1].trim());
        }
    }
    return null; // zwróć zbudowany obiekt
}
```

4.2 Parsowanie liczb — pułapka z przecinkiem

Polskie pliki CSV używają przecinka jako separatora dziesiętnego (np. 1,23). Java wymaga kropki. Zawsze zamieniaj przed parsowaniem:

```
// ŹLE – rzuci NumberFormatException dla "1,23"
double v = Double.parseDouble("1,23");

// DOBRZE – zamień przecinek na kropkę
double v = Double.parseDouble("1,23".replace(",", "."));
```

4.3 Parsowanie dat

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

// Format US: "1/5/21" (kolokwium 2021)
LocalDate d = LocalDate.parse("1/5/21",
    DateTimeFormatter.ofPattern("M/d/yy"));

// Format PL: "05.01.21"
LocalDate d = LocalDate.parse("05.01.21",
    DateTimeFormatter.ofPattern("d.MM.yy"));

// Formatowanie daty do napisu
String s = d.format(DateTimeFormatter.ofPattern("d.MM.yy"));
```

4.4 Files.readAllLines — prostsze od Scannera

```
import java.nio.file.Files;
import java.nio.file.Path;
```

```
List<String> lines = Files.readAllLines(Path.of("plik.csv"));

String header = lines.get(0);          // pierwsza linia
String[] cols = header.split(";", -1); // podziel

for (int i = 2; i < lines.size(); i++) { // od 3. linii
    String[] parts = lines.get(i).split(";", -1);
    // przetwarzaj...
}
```

■ Zawsze używaj `split(";", -1)` z parametrem `-1`. Bez niego Java ignoruje puste kolumny na końcu linii — to może powodować błąd.

5. Własne wyjątki

W każdym kolokwium jest próba o własny wyjątek. Są dwa rodzaje — wybór zależy od treści zadania.

5.1 Checked vs Unchecked

	Checked Exception	Unchecked Exception
Dziedziczy po	Exception	RuntimeException
Kompilator wymaga	throws lub try-catch	Nic (opcjonalnie)
Kiedy używa	Brak pliku, nie znaleziono kraju	Błąd logiki, zły argument
Przykład z kolokwium	CountryNotFoundException (2021) AmbiguousProductException (2022)	RuntimeException (2023) IllegalArgumentException (2024)

5.2 Jak napisać własny wyjątek

```
// Checked – kompilator wymusi obsługę
class CountryNotFoundException extends Exception {
    public CountryNotFoundException(String countryName) {
        super(countryName); // getMessage() zwróci countryName
    }
}

// Unchecked – nie trzeba deklarować throws
class CityNotOnLandException extends RuntimeException {
    public CityNotOnLandException(String cityName) {
        super(cityName);
    }
}
```

5.3 Rzucanie i przechwytywanie

```
// RZUCANIE
void someMethod(String country) throws CountryNotFoundException {
    if (!found) {
        throw new CountryNotFoundException(country);
    }
}

// PRZECHWYTYWANIE i pomijanie (krok 6 kolokwium 2021)
List<Country> result = new ArrayList<>();
for (String name : names) {
    try {
        result.add(Country.fromCsv(name));
    } catch (CountryNotFoundException e) {
        System.out.println(e.getMessage()); // wypisz nazwę i pomiń
    }
}

// assertThrows w teście (JUnit)
RuntimeException ex = assertThrows(RuntimeException.class, () -> {
    land.addCity(cityOnWater);
});
assertEquals("Nazwa Miasta", ex.getMessage());
```


6. Sortowanie list

W każdym kolokwium jest sortowanie listy obiektów wg jakiegoś kryterium. Używa się do tego Comparator.

6.1 Podstawowe wzorce

```
// Rosnąco po nazwie (String)
list.sort(Comparator.comparing(Country::getName));

// Malejąco po liczbie całkowitej
list.sort(Comparator.comparingInt(Country::getDeaths).reversed());

// Malejąco po double
list.sort(Comparator.comparingDouble(City::getDiff).reversed());

// Własna funkcja klucza (np. suma z zakresu dat)
list.sort(Comparator.comparingInt((Country c) -> {
    int sum = 0;
    // oblicz sumę zgonów w przedziale dat
    return sum;
}).reversed());
```

6.2 Komparator jako metoda statyczna (kolokwium 2024)

Czasem trzeba prosić, aby metoda dała się użyć jako komparator. Wystarczy, aby przyjmowała dwa obiekty tego samego typu i zwracała int:

```
// W klasie City:
public static int worstTimezoneFit(City a, City b) {
    // Wiksza różnica → wcześniej (na początku listy)
    return Double.compare(
        Math.abs(b.getTimeDiff()),
        Math.abs(a.getTimeDiff())
    );
}

// Użycie:
List<City> cities = new ArrayList<>(...);
cities.sort(City::worstTimezoneFit); // referencja do metody
```

Jeśli chcesz sortować malejąco przez własny komparator — zamień kolejność a i b, albo dodaj .reversed() ale to nie zadziała z referencją do metody.

7. Zapis do pliku

Ostatni krok (lub jeden z ostatnich) to zazwyczaj zapisanie danych do pliku tekstowego lub SVG.

7.1 Zapis tekstu z tabulatorami (kolokwium 2021)

```
import java.io.PrintWriter;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.format.DateTimeFormatter;

void saveToDataFile(Path outputPath) throws IOException {
    try (PrintWriter pw = new PrintWriter(Files.newBufferedWriter(outputPath))) {
        DateTimeFormatter fmt = DateTimeFormatter.ofPattern("d.MM.yy");

        for (LocalDate date : allDates) {
            pw.printf("%s\t%d\t%d%n",
                date.format(fmt),
                getConfirmedCases(date),
                getDeaths(date)
            );
        }
    } // try-with-resources automatycznie zamyka plik
```

7.2 Tworzenie katalogu i zapis SVG (kolokwium 2024)

```
import java.nio.file.Files;
import java.nio.file.Path;

// Stwórz katalog (i po■rednie katalogi je■li nie istniej■)
Path dir = Path.of("12:00:00"); // nazwa = toString() zegara
Files.createDirectories(dir);

// Zapisz plik dla ka■dego miasta
for (City city : cities) {
    clock.setCity(city);
    String svg = clock.toSvg();
    Files.writeString(dir.resolve(city.getName() + ".svg"), svg);
}
```

7.3 Sprawdzenie pliku przed otwarciem

```
import java.io.FileNotFoundException;

static void setFiles(String casesPath, String deathsPath)
    throws FileNotFoundException {
    Path p1 = Path.of(casesPath);
    Path p2 = Path.of(deathsPath);

    if (!Files.exists(p1) || !Files.isReadable(p1))
        throw new FileNotFoundException(casesPath);
    if (!Files.exists(p2) || !Files.isReadable(p2))
        throw new FileNotFoundException(deathsPath);

    // ustaw statyczne pola
    confirmedCasesPath = p1;
    deathsPath = p2;
```

}

8. Testy JUnit 5 (od kolokwium 2023)

Od 2023 roku kolokwia wymagają pisania testów JUnit. To nie jest trudne — wystarczy znać kilka adnotacji i metod.

8.1 Podstawowy test

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class PolygonTest {

    @Test
    void pointInsidePolygon() {
        Polygon polygon = new Polygon(List.of(
            new Point(0, 0), new Point(10, 0),
            new Point(10, 10), new Point(0, 10)
        ));

        assertTrue(polygon.inside(new Point(5, 5))); // punkt wewnątrz
    }

    @Test
    void pointBelowPolygon() {
        // ... stwórz wielokąt
        assertFalse(polygon.inside(new Point(5, 15))); // punkt pod wielokątem
    }

    @Test
    void pointRightOfPolygon() {
        assertFalse(polygon.inside(new Point(15, 5))); // punkt na prawo
    }
}
```

8.2 Test wyjątku

```
@Test
void addCityNotOnLandThrowsWithCityName() {
    Land land = new Land(List.of(/* punkty lądu */));
    City city = new City(new Point(100, 100), "Atlantyda", 5.0);
    // punkt (100,100) jest poza lądem

    RuntimeException ex = assertThrows(RuntimeException.class, () -> {
        land.addCity(city);
    });

    assertEquals("Atlantyda", ex.getMessage());
}
```

8.3 Test parametryzowany (wiele przypadków naraz)

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.MethodSource;
import java.util.stream.Stream;

class CityTest {
    static City portCity = new City(...); // pole klasy
    static City inlandCity = new City(...);
}
```

```

@ParameterizedTest
@MethodSource("resourceCases")
void testAddResource(City city, Resource resource, boolean expected) {
    city.addResourcesInRange(List.of(resource), 1000.0);
    assertEquals(expected, city.resources.contains(resource.type()));
}

static Stream<Arguments> resourceCases() {
    return Stream.of(
        Arguments.of(inlandCity, new Resource(nearPt, Resource.Type.Coal), true),
        Arguments.of(inlandCity, new Resource(farPt, Resource.Type.Wood), false),
        Arguments.of(portCity, new Resource(seaPt, Resource.Type.Fish), true),
        Arguments.of(inlandCity, new Resource(seaPt, Resource.Type.Fish), false)
    );
}

```

■ Jeśli nie wiesz jak zrobić `@ParameterizedTest` — napisz zwykłe `@Test` dla każdego przypadku. Dostaniesz cztery punkty.

9. Geometria i SVG (kolokwia 2023 i 2024)

9.1 Record Point

// Record to skrót – kompilator sam generuje konstruktor, gettery, equals, toString
record Point(double x, double y) {

```
    // Możesz dodać własne metody
    double distanceTo(Point other) {
        return Math.sqrt(
            Math.pow(this.x - other.x, 2) +
            Math.pow(this.y - other.y, 2)
        );
    }
}
```

```
// Użycie (gettery przez nazwę pola bez "get"):
Point p = new Point(3.0, 4.0);
double x = p.x();    // nie p.getX()!
double y = p.y();
```

9.2 Algorytm ray casting — punkt w wielokącie

Algorytm z kolokwium 2023. Rzuci promień z punktu w lewo i liczy ile razy przecina krawędzie wielokąta.
Nieparzysta liczba = punkt wewnątrz.

```
public boolean inside(Point point) {
    int counter = 0;
    int n = points.size();

    for (int i = 0; i < n; i++) {
        Point pa = points.get(i);
        Point pb = points.get((i + 1) % n); // ostatni łączy z pierwszym

        // Upewnij się, że pa jest niżej niż pb
        if (pa.y() > pb.y()) {
            Point tmp = pa; pa = pb; pb = tmp;
        }

        // Sprawdź, czy punkt leży w pionowym zakresie krawędzi
        if (pa.y() < point.y() && point.y() < pb.y()) {
            double d = pb.x() - pa.x();
            double x;
            if (d == 0) {
                x = pa.x();
            } else {
                double a = (pb.y() - pa.y()) / d;
                double b = pa.y() - a * pa.x();
                x = (point.y() - b) / a;
            }
            if (x < point.x()) counter++;
        }
    }
    return counter % 2 == 1;
}
```

9.3 Kwadrat z centrum (klasa City, 2023)

```
// Konstruktor City przyjmuje: centrum, nazwę, długość boku muru
public City(Point center, String name, double wallLength) {
    super(squareVertices(center, wallLength)); // super = Polygon
    this.center = center;
    this.name = name;
}

// Prywatna metoda pomocnicza – oblicza wierzchołki kwadratu
private static List<Point> squareVertices(Point c, double side) {
    double h = side / 2.0;
    return List.of(
        new Point(c.x() - h, c.y() - h), // lewy dolny
        new Point(c.x() + h, c.y() - h), // prawy dolny
        new Point(c.x() + h, c.y() + h), // prawy górny
        new Point(c.x() - h, c.y() + h)  // lewy górny
    );
}
```

9.4 Generowanie SVG (kolokwium 2024)

```
public String toSvg() {
    StringBuilder sb = new StringBuilder();
    sb.append("<svg xmlns=\"http://www.w3.org/2000/svg\" width=\"400\" height=\"400\">\n");

    // Tarcza zegara
    sb.append("<circle cx=\"200\" cy=\"200\" r=\"190\" ");
    sb.append("fill=\"white\" stroke=\"black\" stroke-width=\"4\"/>\n");

    // Wskazówka – obrócona o angle stopni wokół centrum
    double angle = secondAngle(time.getSecond());
    sb.append("<line x1=\"200\" y1=\"200\" x2=\"200\" y2=\"50\" ");
    sb.append("stroke=\"red\" stroke-width=\"2\" ");
    sb.append(String.format("transform=\"rotate(%.2f 200 200)\"/>\n", angle));

    sb.append("</svg>");
    return sb.toString();
}

// Kąty wskazówek (obróty = stopnie zgodnie z ruchem wskazówek)
double secAngle = second * 6.0; // 360 / 60 = 6°/s
double minAngle = minute * 6.0 + second * 0.1; // płynny ruch
double hourAngle = (hour % 12) * 30.0 + minute * 0.5; // płynny ruch
```

10. Operacje na czasie (kolokwium 2024)

10.1 LocalTime — podstawy

```
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;

// Bieżący czas
LocalTime now = LocalTime.now();

// Utwórz z godziny, minuty, sekundy
LocalTime t = LocalTime.of(13, 30, 45);

// Pobierz składowe
int h = t.getHour();    // 13
int m = t.getMinute();  // 30
int s = t.getSecond();  // 45

// Dodaj/odejmij
LocalTime later = t.plusHours(2);    // 15:30:45
LocalTime before = t.minusHours(2);  // 11:30:45

// Format
String s24 = t.format(DateTimeFormatter.ofPattern("HH:mm:ss")); // 13:30:45
```

10.2 Format 12h z AM/PM

```
String format12h(LocalTime t) {
    int hour = t.getHour();
    int minute = t.getMinute();
    int second = t.getSecond();

    String ampm = hour < 12 ? "AM" : "PM";
    int h12 = hour % 12;
    if (h12 == 0) h12 = 12; // 0:xx → 12:xx AM, 12:xx → 12:xx PM

    // Godzina BEZ zera wiodącego (1:00, nie 01:00)
    return String.format("%d:%02d:%02d %s", h12, minute, second, ampm);
}

// Testy:
// 00:00:00 → "12:00:00 AM"    12:00:00 → "12:00:00 PM"
// 01:00:00 → "1:00:00 AM"    13:00:00 → "1:00:00 PM"
// 11:59:00 → "11:59:00 AM"   23:59:00 → "11:59:00 PM"
```

10.3 Local Mean Time (czas miejscowy)

Czas miejscowy oblicza się z długości geograficznej. Długość $-180^{\circ}..+180^{\circ}$ odpowiada przesunięciu $-12h..+12h$ względem UTC.

```
LocalTime localMeanTime(LocalTime zoneTime, int timezoneOffset, double longitude) {
    // 1. Przelicz czas strefowy na UTC
    LocalTime utc = zoneTime.minusHours(timezoneOffset);

    // 2. Przelicz długość geograficzną na sekundy przesunięcia
    //    longitude / 180 * 12 godzin = longitude * 240 sekund
    long offsetSeconds = Math.round(longitude * 240.0); // 4 minuty na stopień

    // 3. Dodaj przesunięcie geograficzne
```



```
        return utc.plusSeconds(offsetSeconds);
    }

    // Przykład z treści: Lublin, strefa +2, długość 22.567°
    // Przesunięcie = 22.567 * 240 = 5416 sekund = 1h 30min 16s
    // 12:00:00 - 2h + 1h 30min 16s = 11:30:16
```

11. Streamy i interfejsy funkcyjne (kolokwium 2022)

Strumienie (Streams) to elegancki sposób na filtrowanie i transformację kolekcji. Interfejsy funkcyjne pozwalają przekazywać metody jako argumenty.

11.1 Najczęstsze operacje Stream

```
List<Product> products = // ...

// Filtrowanie
List<Product> cheap = products.stream()
    .filter(p -> p.getPrice() < 10.0)
    .collect(Collectors.toList());

// Mapowanie (transformacja)
List<String> names = products.stream()
    .map(Product::getName) // lub p -> p.getName()
    .collect(Collectors.toList());

// Suma / Średnia
double total = products.stream()
    .mapToDouble(Product::getPrice)
    .sum();

double avg = products.stream()
    .mapToDouble(Product::getPrice)
    .average()
    .orElse(0.0);

// Filtrowanie po prefiksie (kolokwium 2022, getProducts)
String prefix = "Bu";
List<Product> matching = products.stream()
    .filter(p -> p.getName().startsWith(prefix))
    .collect(Collectors.toList());
```

11.2 Interfejs funkcyjny jako parametr metody

```
// Własny interfejs funkcyjny (lub użyj Function<Path, T>)
@FunctionalInterface
interface CsvFactory<T> {
    T create(Path path) throws IOException;
}

// Metoda przyjmująca fabrykę
static <T> void addProducts(CsvFactory<T> factory, Path dir) throws IOException {
    try (var stream = Files.list(dir)) {
        stream.filter(p -> p.toString().endsWith(".csv"))
            .forEach(path -> {
                try {
                    products.add(factory.create(path));
                } catch (IOException e) { throw new RuntimeException(e); }
            });
    }
}

// Wywołanie – przekazujesz METODĘ jako argument
addProducts(NonFoodProduct::fromCsv, Path.of("data/nonfood"));
```

```
addProducts(FoodProduct::fromCsv, Path.of("data/food"));
```

12. Parsowanie XML/SVG z Jacksonem (kolokwium 2023)

W 2023 roku mapa była w formacie SVG (czyli XML). Do parsowania użyto biblioteki Jackson Dataformat XML — już dodana do pom.xml.

12.1 Jak to działa

```
// Klasa reprezentująca element <svg>
static public final class Svg {
    @JacksonXmlElementWrapper(useWrapping = false)
    @JsonProperty("polygon")
    private List<Map<String, String>> polygons; // wszystkie <polygon>

    @JacksonXmlElementWrapper(useWrapping = false)
    @JsonProperty("rect")
    private List<Map<String, String>> rects; // wszystkie <rect>

    @JacksonXmlElementWrapper(useWrapping = false)
    @JsonProperty("text")
    private List<Map<String, String>> texts; // wszystkie <text>
}

// Parsowanie pliku SVG
XmlMapper xmlMapper = new XmlMapper();
Svg svg = xmlMapper.readValue(new File("map.svg"), Svg.class);

// Każdy element to mapa atrybut → wartość
// np. polygon: {"points": "10,20 30,40 50,60", "fill": "green"}
for (Map<String, String> poly : svg.polygons) {
    String points = poly.get("points"); // "10,20 30,40 50,60"
    // parsuj punkty...
}
```

12.2 Parsowanie atrybutu "points" ze SVG polygon

```
// Format: "x1,y1 x2,y2 x3,y3 ..."
List<Point> parsePoints(String pointsAttr) {
    List<Point> result = new ArrayList<>();
    for (String pair : pointsAttr.trim().split("\\s+")) { // podziel po spacjach
        String[] xy = pair.split(",");
        result.add(new Point(
            Double.parseDouble(xy[0]),
            Double.parseDouble(xy[1])
        ));
    }
    return result;
}
```

12.3 Parsowanie rect — centrum miasta

```
// Atrybut "x" i "y" to lewy GÓRNY róg prostokąta (nie centrum!)
// Musisz sam obliczyć centrum

Map<String, String> rectAttrs = // ...
double x = Double.parseDouble(rectAttrs.get("x"));
double y = Double.parseDouble(rectAttrs.get("y"));
double width = Double.parseDouble(rectAttrs.get("width"));
double height = Double.parseDouble(rectAttrs.get("height"));
```

```
// Centrum = lewy_górny + połowa szerokości/wysokości  
Point center = new Point(x + width / 2.0, y + height / 2.0);  
// Długość boku muru = width (zakładając kwadrat)
```

13. Najczęstsze błędy i jak ich unikać

■ NullPointerException

Problem: Wywołanie metody na null. Sprawdź czy nie używasz obiektu zanim go zainicjalizujesz.

■ Rozwiązanie: Zmienna = null. Dodaj null-check: `if (obj != null) { ... }`

■ NumberFormatException

Problem: Parsowanie stringa do liczby gdy string zawiera spację, przecinek lub inny znak.

■ Rozwiązanie: Zawsze: `s.trim().replace(",", ".")` przed `parseDouble()`

■ IndexOutOfBoundsException

Problem: Dostęp do elementu listy/tablicy poza jej rozmiarem.

■ Rozwiązanie: Sprawdź rozmiar przed dostępem. Użyj `i < list.size()`

■ split() bez -1

Problem: `split(",")` ignoruje puste kolumny na końcu linii CSV.

■ Rozwiązanie: Zawsze: `split(";", -1)`

■ Niezamknięty plik

Problem: Scanner/PrintWriter nie jest zamknięty po użyciu — ryzyko utraty danych.

■ Rozwiązanie: Zawsze używaj try-with-resources: `try (Scanner sc = new Scanner(path)) { ... }`

■ super() nie jako pierwsze

Problem: Wywołanie `super()` w konstruktorze musi być PIERWSZĄ instrukcją.

■ Rozwiązanie: `public Subclass() { super(); /* tu reszta */ }`

■ Brak @Override

Problem: Bez `@Override` Java nie informuje gdy nazwa metody jest błędna.

■ Rozwiązanie: Zawsze dodawaj `@Override` przy nadpisywaniu metod.

14. Przydatne one-linery — szybka ■ci■gawka

Operacja	Kod
Suma tablicy Double[]	<code>Arrays.stream(arr).mapToDouble(Double::doubleValue).sum()</code>
■rednia tablicy	<code>Arrays.stream(arr).mapToDouble(Double::doubleValue).average().orElse(0)</code>
Tablica → Lista	<code>new ArrayList<>(Arrays.asList(arr))</code>
Lista → Tablica	<code>list.toArray(new String[0])</code>
Liczba z zerem wiod■cym	<code>String.format("%02d", 5) // → "05"</code>
LocalDate zakres	<code>!date.isBefore(from) && !date.isAfter(to)</code>
LocalDate iteracja po datach	<code>from.datesUntil(to.plusDays(1)).forEach(date -> { ... })</code>
Odleg■o■ ■mi■dzy punktami	<code>Math.sqrt(Math.pow(x2-x1, 2) + Math.pow(y2-y1, 2))</code>
Warto■ ■bezwzgl■dna różnicy czasu	<code>Math.abs(Duration.between(t1, t2).toSeconds())</code>
Path z ■a■cucha	<code>Path.of("data/nonfood/plik.csv")</code>
Listowanie plików w katalogu	<code>Files.list(dir).filter(p -> p.toString().endsWith(".csv"))</code>
String z powtórze■	<code>String.join(", ", list)</code>
Enum do napisu	<code>Resource.Type.Coal.name() // → "Coal"</code>
Napis do Enum	<code>Resource.Type.valueOf("Coal") // → Resource.Type.Coal</code>
Suma w zakresie dat (pattern)	<code>dates.stream().filter(d -> !d.isBefore(from) && !d.isAfter(to)).mapToInt(d -> stats.get(d)).sum()</code>

15. Strategia na dzień egzaminu

0–5 min	Pobierz i otwórz repo, poczekaj na Maven
5–15 min	Przeczytaj CAŁE treści zadania. Zidentyfikuj klasy, wyjątek, dane, metody
15–30 min	Krok 1-2: klasa abstrakcyjna + podklasy (fundament)
30–60 min	Krok 3-5: fromCsv + wyjątek + struktury danych
60–80 min	Krok 6-8: przeciążenia, metody abstrakcyjne, sortowanie
80–90 min	Krok 9+: zapis do pliku, testy JUnit, Jackson, SVG
ostatnie 5 min	Spakuj projekt do ZIP (wyklucz katalog target i .idea)

■ Zrób git commit lub ZIP po KAŻDYM działającym kroku. Lepiej wysłać 5 kroków niż 0.

■ Pakuj projekt do ZIP: Plik → Export → Export to ZIP w IntelliJ, LUB ręcznie: kliknij prawym na katalog projektu → compress. Usuń katalog 'target' i '.idea' — treści kolokwium tego wymaga.

■ Jeśli utkniesz na jakimś kroku — przejdź do następnego. Punkty za późniejsze kroki też się liczą.

Powodzenia! ■